

Agentic AI Ultimate Cheat Sheet

From Chatbots to Autonomous Agents: Frameworks & Patterns

1 What is Agentic AI?

Definition

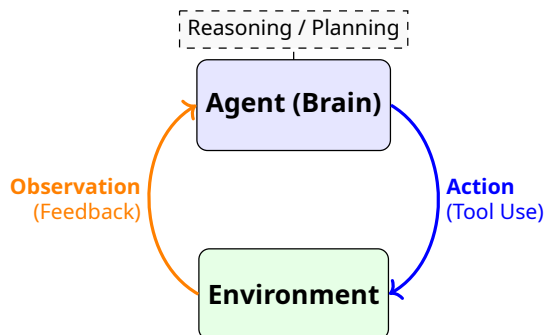
Systems where LLMs actively **reason**, **plan**, and use **tools** to achieve goals autonomously, rather than just responding to prompts.

Chatbot vs. Agent

- **Chatbot:** Passive. Input → Output.
- **Agent:** Active. Goal → Plan → Act → Refine.

The Core Loop

The continuous cycle of reasoning and acting.



2 Agent Anatomy

1. The Brain (LLM)

The core controller (e.g., GPT-4, Claude 3.5 Sonnet) responsible for reasoning and orchestration.

2. Tools (Capabilities)

Functions the agent can execute.

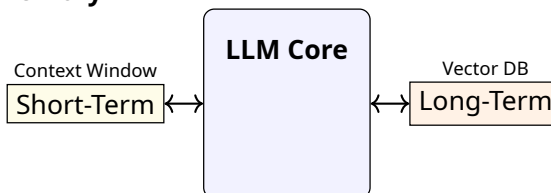
- **Web Search:** Google/Bing APIs.
- **Code Execution:** Python REPL.
- **Custom APIs:** Database queries, Slack hooks.

3. Planning

Breaking complex goals into manageable steps.

- **Decomposition:** Split goal into sub-tasks.
- **Reflection:** Critique past actions.

4. Memory

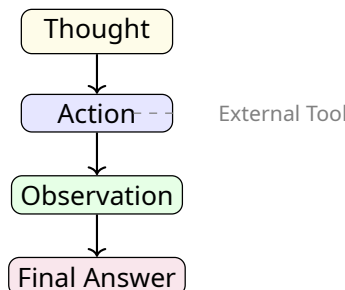


- **Short-term:** Chat history (limited by context).
- **Long-term:** RAG / Vector Databases.

3 Cognitive Architectures

ReAct (Reason + Act)

The standard pattern. The model generates a "Thought", takes an "Action", receives an "Observation", and repeats.



```
Prompt:
Question: Elevation of Colorado?
Thought: Search "Colorado elevation".
Action: Search["Colorado elevation"]
Observation: 6,800 ft
Thought: I have the answer.
Final Answer: 6,800 ft.
```

Plan-and-Solve

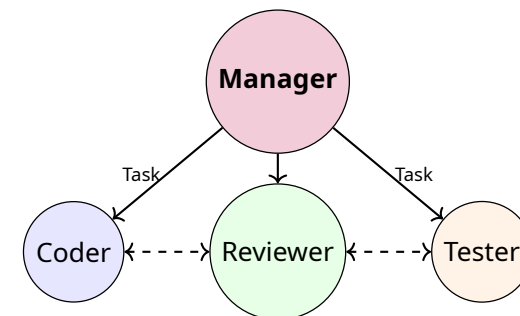
Generate a full plan *first*, then execute it step-by-step. Reduces hallucination in math/logic tasks.

Reflexion

An agent that critiques its own output and retries if it fails validation. "Self-correction".

4 Multi-Agent Systems (MAS)

Specialized agents working together often outperform one generalist agent.



Patterns

- **Hierarchical:** A "Manager" LLM breaks down the task and assigns to "Worker" agents (e.g., Coder, Designer).
- **Sequential Handoff:** Agent A passes state to Agent B (e.g., Research → Write → Edit).
- **Joint Chat:** Agents conversing in a shared thread.

Why MAS?

- **Focus:** Smaller system prompts per agent.
- **Modularity:** Easier to debug specific skills.

5 Frameworks & Tools

LangChain / LangGraph

Standard for Python. **LangGraph** allows defining agents as graphs (nodes=agents, edges=communication).

```
# LangGraph Basic Concept
workflow = StateGraph(AgentState)
workflow.add_node("researcher", agent1)
workflow.add_node("writer", agent2)
workflow.add_edge("researcher", "writer")
app = workflow.compile()
```

AutoGen (Microsoft)

Focuses on multi-agent conversations. Agents automatically chat to solve tasks.

```
# AutoGen Concept
user = UserProxyAgent("user")
coder = AssistantAgent("coder")
user.initiate_chat(coder,
    message="Plot a stock chart")
```

CrewAI

High-level framework for role-playing agents. Good for business workflows.

```
# CrewAI Concept
researcher = Agent(role='Researcher', ...)
writer = Agent(role='Writer', ...)
crew = Crew(agents=[researcher, writer])
crew.kickoff()
```

6 Tool Use (Function Calling)

How models interact with the outside world.

Mechanism 1. User defines tools (JSON Schema).
2. LLM outputs structured JSON (e.g., {"name":

"calc", "args": {...}}). 3. System executes function code. 4. System feeds result back to LLM.

```
tools = [{
    "name": "get_weather",
    "parameters": {
        "type": "object",
        "properties": {
            "city": {"type": "string"}
        }
    }
}]
```

7 Evaluation & Challenges

Metrics

- **Success Rate:** Did it achieve the goal?
- **Steps:** How efficient was the path?
- **Cost:** Token usage per task.

Benchmarks

- **AgentBench:** Evaluating across environments (OS, DB, Knowledge).
- **SWE-bench:** Solving GitHub issues (Coding).
- **GAIA:** General AI Assistants benchmark.

Challenges

- **Infinite Loops:** Agent getting stuck repeating the same thought.
- **Tool Hallucination:** Trying to call tools that don't exist.
- **Error Propagation:** One bad step ruins the whole chain.